

US Census IV&V
Independent Verification of Census Bureau Data Releases

Troy Altus

2025-12-31

Table of contents

1	How to Read a Census Release	1
2	How to Read a Census Release	3
2.1	The Press Release Pipeline	3
2.2	The Three Main Surveys	3
2.3	The Verification Workflow	4
2.4	The Census API in Practice	4
2.5	Summary	5
2.6	Further Reading	5
3	Preface	7
	Preface	9
	What This Is	9
	The Census API	9
	How to Add a New Note	10
4	Education Enrollment, 2024	11
5	Education Enrollment, 2024	13
5.1	The Claim	13
5.2	The Source	13
5.3	The API Pull	14
5.4	Comparison	15
5.5	Observations	15
5.6	Summary	16
	References	17
	Appendices	19
A	Census API Utilities	19
	Census API Utilities	21
	Core API Wrapper	21
	ACS Education Variables (Reference)	23
	Weighted Estimate Helper	24

Chapter 1

How to Read a Census Release

Chapter 2

How to Read a Census Release

Learning Objectives

- Understand how Census Bureau press releases are structured and what they typically claim
- Know the difference between ACS, CPS, and Decennial Census data products
- Understand the Census API well enough to pull a table and compare it to a headline figure

2.1 The Press Release Pipeline

The Census Bureau publishes hundreds of data releases each year. The public-facing version is a press release with a headline, a few bullet points, and a link to detailed tables. The statistical version is a set of tables, microdata files, and an API endpoint that returns the underlying estimates.

The headline figure is always accurate in the sense that it correctly describes what the tables say. What the press release does not always make clear is which survey produced the number, what population was counted, what years are being compared, and what the margin of error is. Those details are in the technical documentation, and they matter.

A release that says “college enrollment reached X million” might be using the October CPS supplement (household survey, smaller sample, asks about current enrollment) or the ACS (larger sample, asks about enrollment in the past 12 months). These are not the same question. The difference is rarely explained in the press release and is always explained in the methodology.

2.2 The Three Main Surveys

Understanding which survey produced a number is the first step in verifying it.

American Community Survey (ACS) Annual survey of approximately 3.5 million households. The gold standard for detailed demographic and economic characteristics. Published in one-year (geographies 65,000 population) and five-year (all geographies) editions. Variables include

education, income, housing, employment, language, citizenship. API endpoint: `/data/acs/acs1` and `/data/acs/acs5`.

Current Population Survey (CPS) Monthly household survey conducted jointly by the Bureau of Labor Statistics and Census. The primary source for labor force statistics. Supplemental surveys run annually on specific topics: school enrollment (October), income and poverty (March), voting (November). Smaller sample than ACS; better for tracking month-to-month change. API endpoint: `/data/cps/<supplement>/<month>`.

Decennial Census Full count (or near-full count) every ten years. Last conducted 2020. Used for apportionment, redistricting, and as a benchmark for survey weighting. API endpoint: `/data/dec/`.

Most education enrollment figures in Census press releases come from the CPS October School Enrollment supplement.

2.3 The Verification Workflow

Each note in this series follows the same sequence:

1. CLAIM - what the press release said, verbatim
2. SOURCE - which survey, which year, which variable group
3. PULL - API call that retrieves the relevant estimate
4. COMPARE - press release figure vs. API result
5. OBSERVE - what the data can do beyond the headline

Step 4 should reproduce the headline figure exactly. If it does not, the discrepancy is documented and explained (usually it is a rounding convention or a population restriction that the press release does not spell out). The interesting work is step 5: what else does this data product contain, what geographies can be disaggregated, what time series goes back how far, and what follow-on questions are worth pursuing.

2.4 The Census API in Practice

```
import requests
import os
import pandas as pd

def census_get(dataset, year, variables, geography, api_key=None):
    """
    Minimal Census API wrapper. Returns a DataFrame.

    Parameters
    -----
    dataset      : str    e.g. "acs/acs1", "cps/school/oct"
    year         : int     e.g. 2024
    variables    : list    e.g. ["NAME", "B15003_001E"]
    geography    : str     e.g. "us:1", "state:*"
    api_key      : str     defaults to CENSUS_API_KEY env var
```



```

"""
key = api_key or os.getenv("CENSUS_API_KEY", "")
base = f"https://api.census.gov/data/{year}/{dataset}"
params = {
    "get": ",".join(variables),
    "for": geography,
    "key": key,
}
r = requests.get(base, params=params, timeout=30)
r.raise_for_status()
data = r.json()
return pd.DataFrame(data[1:], columns=data[0])

def list_variables(dataset, year):
    """
    Print available variable groups for a dataset/year.
    Useful for exploration.
    """
    url = f"https://api.census.gov/data/{year}/{dataset}/groups.json"
    r = requests.get(url, timeout=30)
    r.raise_for_status()
    groups = r.json().get("groups", [])
    df = pd.DataFrame(groups)[["name", "description"]].sort_values("name")
    return df

```

💡 API Key Setup

The Census API key is stored in macOS Keychain and loaded by `~/secrets/load_keys.sh` at login. In Python: `os.getenv("CENSUS_API_KEY")`. If running on a new machine, get a free key at api.census.gov/data/key_signup.html.

2.5 Summary

Census press releases present accurate but compressed descriptions of survey results. Verifying them requires knowing which survey produced the number, locating the variable in the API, and pulling the estimate directly. The three main sources are the ACS, the CPS, and the Decennial Census; most education and labor statistics in press releases come from CPS supplements. Each note in this series walks through that verification workflow for one release, with observations on what the underlying data product can support beyond the headline.

2.6 Further Reading

- [Census Bureau API documentation](#) — complete list of available datasets and variables
- [Census Academy](#) — free training on using Census data products
- Citro, C.F. & Michael, R.T. (eds.) *Measuring Poverty: A New Approach*. National

Academies Press, 1995. Background on how survey-based poverty and enrollment estimates are constructed.

Chapter 3

Preface

Preface

What This Is

The US Census Bureau sends email announcements when new data releases go out — surveys completed, tables published, headline figures highlighted. This series takes those announcements at face value and then checks them.

The goal is not adversarial. The Bureau is a competent statistical agency and its headline figures are generally accurate. The goal is to understand *how* the numbers were produced: which survey, which variables, which population definition, which suppression rules, and what the API returns when you ask the same question the press release answered. That understanding is worth having on its own terms, and it surfaces questions worth pursuing as standalone projects.

Each note follows the same structure: the claim, the source, the API pull, the comparison, and observations about what the data can and cannot do. Notes accumulate as the emails arrive.

The Census API

The US Census Bureau exposes most of its data through a public REST API at `api.census.gov`. An API key is required but free. The key used in this analysis is stored in macOS Keychain and loaded into the shell environment; code accesses it via `os.getenv("CENSUS_API_KEY")`.

The main endpoints used here are:

Endpoint	Dataset	Notes
<code>/data/acs/acs1</code>	American Community Survey 1-year	Annual, geography 65K pop
<code>/data/acs/acs5</code>	American Community Survey 5-year	Annual, all geographies
<code>/data/cps/school/oct</code>	Current Population Survey — School Enrollment	October supplement
<code>/data/dec/dp</code>	Decennial Census Data Profiles	2020 onward

Variable names, group codes, and geography parameters are documented at `api.census.gov/data/<year>/<data>`. The API is not always intuitive. Part of what these notes capture is what you learn navigating it.

How to Add a New Note

When a new Census email arrives worth checking:

1. Add a new `.qmd` file: `NN-<topic>-<year>.qmd`
2. Register it in `_quarto.yml` under `chapters`:
3. Use the template structure from any existing note: `claim → source → pull → compare → observations`
4. Run `pixi run render` and push

The bibliography accumulates in `references.bib`. API utility functions accumulate in `appendix-a-code.qmd`.

Troy Altus 2026

Chapter 4

Education Enrollment, 2024

Chapter 5

Education Enrollment, 2024

This Note

Release: US Census Bureau email announcement, May 2026 **Survey:** Current Population Survey — October School Enrollment Supplement, 2024 **Claim:** [paste headline figure from the email here] **Verdict:** [fill in after pulling data]

5.1 The Claim

[Paste the exact headline text from the Census Bureau email here.]

Source: Census Bureau press release / email announcement, [date]

The figures typically reported in school enrollment releases include: - Total enrolled population (all ages) - Nursery school / preschool enrollment - K–12 enrollment - College enrollment (2-year and 4-year) - Year-over-year change

The key question for each figure: which population definition, which reference period, and which variable group in the CPS October supplement produced it?

5.2 The Source

The October CPS School Enrollment supplement asks respondents whether they are currently enrolled in school. The reference period is the week containing the survey date (October). This differs from the ACS education enrollment question, which asks about enrollment at any time in the past 12 months — a broader definition that produces higher enrollment counts.

Survey: Current Population Survey — School Enrollment (October) **Year:** 2024 **API dataset:** cps/school/oct **Key variables:** HEFAMINC, PESCHFT, PEEDUCA, PESCHLVL

The CPS Public Use Microdata (PUMD) is the most granular option. The API provides tabulated estimates. Both should reproduce the headline.

5.3 The API Pull

TODO — Fill In After Pull

Run the cells below after the 2024 CPS October supplement is available via the API (typically released ~6 months after the reference month). Check availability at api.census.gov/data/<year>/cps/school/oct/variables.json.

```
import os
import requests
import pandas as pd

API_KEY = os.getenv("CENSUS_API_KEY", "")
YEAR = 2024
DATASET = "cps/school/oct"

# Check available variables for this dataset/year
var_url = f"https://api.census.gov/data/{YEAR}/{DATASET}/variables.json"
r = requests.get(var_url, timeout=30)

if r.status_code == 200:
    variables = r.json()["variables"]
    # Filter to school enrollment variables
    enroll_vars = {k: v for k, v in variables.items()
                    if "ENROLL" in k.upper() or "SCHOOL" in k.upper()
                    or "PESC" in k.upper()}
    for k, v in sorted(enroll_vars.items()):
        print(f"{k:25s} {v.get('label', '')}")
else:
    print(f"Dataset not yet available for {YEAR}: HTTP {r.status_code}")

# National enrollment by school level
# PESCHLVL: school level (nursery, K-12, college)
# PESCHFT: full-time vs part-time college enrollment

params = {
    "get": "PESCHLVL,PWSSWGT", # school level + person weight
    "for": "us:1",
    "key": API_KEY,
}

base = f"https://api.census.gov/data/{YEAR}/{DATASET}"
r = requests.get(base, params=params, timeout=30)
r.raise_for_status()
data = r.json()
df = pd.DataFrame(data[1:], columns=data[0])

# Convert to numeric
```

```

df["PESCHLVL"] = pd.to_numeric(df["PESCHLVL"])
df["PWSSWGT"] = pd.to_numeric(df["PWSSWGT"])

# School level codes (check variables.json for exact mapping)
level_map = {
    1: "Nursery/preschool",
    2: "K-12",
    3: "College/university",
    -1: "Not enrolled / not applicable",
}

df["level"] = df["PESCHLVL"].map(level_map)

totals = (df[df["PESCHLVL"] > 0]
          .groupby("level")["PWSSWGT"]
          .sum()
          .div(1000) # to thousands
          .reset_index()
          .rename(columns={"PWSSWGT": "enrolled_thousands"}))

print(totals.to_string(index=False))

```

5.4 Comparison

Figure	Press Release	API Result	Difference
Total enrolled	—	—	—
K-12	—	—	—
College	—	—	—

Fill in after running the pull above.

5.5 Observations

Things to note while pulling this data — potential seeds for future projects:

- 1. ACS vs. CPS enrollment gap** The ACS B14001 group reports school enrollment for the past 12 months; the CPS October supplement reports current enrollment. The ACS count is consistently higher. Quantifying this gap by level and geography is a clean methodological note.
- 2. Geographic disaggregation** The CPS is designed for national and state-level estimates. For county or tract-level enrollment, the ACS 5-year is the right product. Mapping K-12 enrollment rates at the county level from ACS 5-year data (B14003) is a natural visualization project.
- 3. Enrollment trends** The CPS October supplement goes back to 1947. A time-series of college enrollment rates by age group and sex is available with no additional data acquisition

beyond API calls.

4. College enrollment by institution type The CPS asks whether enrolled at 2-year or 4-year institution (PESCHLVL values distinguish these). Tracking the 2-year vs. 4-year split over time, especially post-2020, may be worth a dedicated note.

5.6 Summary

[Fill in after completing the pull and comparison.]

The Census Bureau’s 2024 education enrollment figures come from the CPS October School Enrollment supplement. The API reproduces the headline [figure / with the following discrepancy]. The dataset supports geographic disaggregation to the state level and time-series analysis back to [year]. Observations on API structure and potential follow-on analyses are logged above.

References

Appendix A

Census API Utilities

Census API Utilities

Reusable functions for pulling and processing Census Bureau data. All functions use `eval: false` — import from this appendix or copy into a note's code cells.

Core API Wrapper

```
import os
import requests
import pandas as pd

def census_get(dataset, year, variables, geography, api_key=None):
    """
    Pull a Census API table. Returns a DataFrame.

    Parameters
    -----
    dataset      : str      e.g. "acs/acs1", "cps/school/oct", "dec/dp"
    year         : int       e.g. 2024
    variables    : list      e.g. ["NAME", "B15003_001E"]
    geography    : str       e.g. "us:1", "state:*", "county:*&in=state:45"
    api_key      : str       defaults to CENSUS_API_KEY env var

    Returns
    -----
    pd.DataFrame with numeric columns coerced where possible

    Examples
    -----
    # National college enrollment from ACS 1-year
    df = census_get("acs/acs1", 2023, ["NAME", "B14001_008E"], "us:1")

    # State-level K-12 enrollment
    df = census_get("acs/acs5", 2023,
                    ["NAME", "B14001_003E", "B14001_004E"],
                    "state:*")
    """
```

```

key = api_key or os.getenv("CENSUS_API_KEY", "")
base = f"https://api.census.gov/data/{year}/{dataset}"
params = {"get": ", ".join(variables), "for": geography, "key": key}
r = requests.get(base, params=params, timeout=30)
r.raise_for_status()
data = r.json()
df = pd.DataFrame(data[1:], columns=data[0])
# Coerce numeric columns
for col in df.columns:
    if col not in ("NAME", "state", "county", "us"):
        df[col] = pd.to_numeric(df[col], errors="ignore")
return df

def census_variables(dataset, year, filter_str=None):
    """
    List available variables for a dataset/year.

    Parameters
    -----
    dataset      : str      e.g. "acs/acs1"
    year         : int
    filter_str   : str      optional substring filter on variable name or label

    Returns
    -----
    pd.DataFrame with columns: name, label, concept, predicateType
    """
    url = f"https://api.census.gov/data/{year}/{dataset}/variables.json"
    r = requests.get(url, timeout=30)
    r.raise_for_status()
    raw = r.json()["variables"]
    rows = [{"name": k, **{f: v.get(f, "") for f in
        ["label", "concept", "predicateType"]}}
        for k, v in raw.items()]
    df = pd.DataFrame(rows).sort_values("name").reset_index(drop=True)
    if filter_str:
        mask = (df["name"].str.contains(filter_str, case=False) |
            df["label"].str.contains(filter_str, case=False) |
            df["concept"].str.contains(filter_str, case=False))
        df = df[mask].reset_index(drop=True)
    return df

def census_groups(dataset, year):
    """
    List available variable groups (table prefixes) for a dataset/year.

```

```

"""
url = f"https://api.census.gov/data/{year}/{dataset}/groups.json"
r = requests.get(url, timeout=30)
r.raise_for_status()
groups = r.json().get("groups", [])
return pd.DataFrame(groups)[["name", "description"]].sort_values("name")

```

ACS Education Variables (Reference)

```

# Commonly used ACS education enrollment groups
ACS_EDUCATION = {
    # School enrollment by level (universe: population 3+ years)
    "B14001": "School enrollment by level of school",
    # B14001_001E Total
    # B14001_002E Enrolled in school
    # B14001_003E Enrolled in nursery school, preschool
    # B14001_004E Enrolled in kindergarten
    # B14001_005E Enrolled in grade 1 to grade 4
    # B14001_006E Enrolled in grade 5 to grade 8
    # B14001_007E Enrolled in grade 9 to grade 12
    # B14001_008E Enrolled in college, undergraduate years
    # B14001_009E Graduate or professional school

    # Educational attainment (population 25+)
    "B15003": "Educational attainment for population 25+",
    # B15003_001E Total
    # B15003_017E Regular high school diploma
    # B15003_022E Bachelor's degree
    # B15003_023E Master's degree
    # B15003_025E Doctorate degree

    # School enrollment by age and sex (more granular)
    "B14003": "Sex by school enrollment by type of school by age",
}

# CPS October supplement - school enrollment variables
CPS_SCHOOL = {
    "PESCHLVL": "School level (1=nursery, 2=K-12, 3=college)",
    "PESCHFT": "Full-time/part-time college enrollment",
    "PEEDUCA": "Highest level of education completed",
    "HEFAMINC": "Family income (recode)",
    "PWSSWGT": "Final person weight (use for population estimates)",
    "PRTAGE": "Age",
    "PESEX": "Sex (1=male, 2=female)",
    "PTDTRACE": "Race (detailed)",
    "PEHSPNON": "Hispanic origin",

```

```
}
```

Weighted Estimate Helper

```
def weighted_total(df, value_col, weight_col, group_col=None):
    """
    Compute weighted population total from CPS microdata.

    Parameters
    -----
    df          : pd.DataFrame  CPS microdata
    value_col   : str           column to filter on (e.g. "PESCHLVL")
    weight_col  : str           person weight column (e.g. "PWSSWGT")
    group_col   : str           optional groupby column

    Returns
    -----
    pd.Series or float: weighted total(s) in persons (divide by 1000 for thousands)
    """
    if group_col:
        return df.groupby(group_col)[weight_col].sum()
    return df[weight_col].sum()
```